TP : Configuration d'un Pipeline CI/CD pour une Application Web PHP

Projet pour le cours : Protéger les Appli. modernes, DevSecOps, Sécurités







Fiona Cuvilliez
Luca Alecu
Daniel Carriba Nosrati
Bence Di Placido

Master 2 Informatique
Parcours Ingénierie Informatique
Université Côte d'Azur

Semestre 1 2026/2027

Sommaire

0 Introduction **2**

1 Compréhension et configuration de base **2**
1.1 Analyse du projet PHP 2
1.2 Configuration Docker 2

2 Mise en place du pipeline CI/CD **2**
2.1 Configuration Github et CircleCI 2
2.2 Création du pipeline CI/CD 3
2.3 Gestion des secrets avec Infisical 3

3 Extension du Pipeline **4**
3.1 Ajout de jobs d'évaluation de code 4
3.2 Intégration de la qualité du code 4
3.3 Déploiement automatisé sur AWS EC2 7

4 Documentation et Rapport **9**

5 Sécurité à tous les étages **11**
5.1 Sécurisation du dépôt Git 11
5.2 Vérification de la fiabilité et de la sécurité de l'image Docker 12
5.3 Sécurisation de l'instance AWS EC2 14

6 Objectifs futurs (bonus) **15**

0 Introduction

Ce rapport a pour but de décrire le travail réaliser dans le cadre du TP *Configuration d'un Pipeline CI/CD pour une Application Web PHP*. Il présentera les différents choix et actions effectués pour les différentes étapes de la configuration du *Pipeline CI/CD*.

1 Compréhension et configuration de base

1.1 Analyse du projet PHP

- Le fichier **composer.json** définit les dépendances du projet PHP ainsi que les contraintes de versions des packages nécessaires à son fonctionnement.
- Le fichier **composer.lock** enregistre les versions exactes des dépendances installées. Il garantit ainsi que tous les environnements utilisent les mêmes versions des packages, notamment lors de l'exécution de **composer install**.
- Le projet est une application PHP simple. Son exécution commence dans **public/index.php**, qui charge l'autoloader de Composer puis utilise **ImageCreator.php**. Cette classe lit les variables d'environnement nécessaires à son exécution et génère une image PNG avec GD et la police consolas.ttf.
- Le dossier **tests/** contient les tests automatisés permettant de vérifier le bon fonctionnement de l'application.
- Le dossier **src/** contient le code source de l'application.

1.2 Configuration Docker

Le Dockerfile part de l'image php:8.2-apache puis :

- installe les bibliothèques système et SQLite
- installe xdebug et les extensions PHP (gd, pdo, zip...)
- installe la CLI Infisical et Composer (pour que la CLI Infisical soit bien installé, nous avions dû modifier l'url initiale par une nouvelle url, à jour.)
- active le mod rewrite d'Apache et fait pointer le site sur le répertoire public
- copie le code, lance composer install (pour installer les dépendances PHP) et règle les droits.
- enfin, expose le port 80

2 Mise en place du pipeline CI/CD

2.1 Configuration Github et CircleCI

Afin de mettre en place le *Pipeline CircleCI*, nous avons créé un répo GitHub avec le projet, ensuite nous avons créé un compte CircleCI et l'avons connecté au répo du TP avec le compte github. CircleCI utilise alors automatiquement le fichier *.circleci/config.yml* déjà présent et qui défini les étapes (jobs) du *pipeline*.

2.2 Création du pipeline CI/CD

La configuration du pipeline CI/CD dans *.circleci/config.yml* a nécessité une harmonisation de l'environnement PHP. La version a été mise à niveau de PHP 8.1 vers PHP 8.2, utilisée dans *composer.json*, les exécuteurs CircleCI (`cimg/php:8.2`) et le Dockerfile (`php:8.2-apache`). Cette uniformisation garantit la compatibilité entre les environnements de développement, de test et de production.

Le pipeline, basé sur CircleCI 2.1, s'organise autour de plusieurs jobs. Le job `build-setup` récupère le code source, installe les dépendances avec Composer et sauvegarde le dossier `vendor` dans le cache et le workspace afin qu'il soit réutilisé par les jobs suivants. Ceux-ci exécutent les tests unitaires avec PHPUnit, analysent le code à l'aide de PHP_CodeSniffer et vérifient les dépendances PHP. Les résultats sont conservés sous forme d'artefacts CircleCI.

La construction de l'image est assurée par `build-docker-image`, qui utilise Docker pour construire l'image à partir de `Docker/Dockerfile`, puis s'authentifie auprès de GitHub Container Registry (GHCR) pour la publier. L'image contient notamment PHP, Apache, Composer, les extensions nécessaires et Infisical CLI. Elle est publiée sur `ghcr.io/bence-dp/devsecopspipeline` avec deux tags : un basé sur le nom de la branche et un autre sur `CIRCLE_SHA1`, permettant d'identifier précisément le commit correspondant et de faciliter le redéploiement.

L'authentification repose sur deux variables configurées dans les paramètres CircleCI : `GHCR_USERNAME`, qui identifie le compte GitHub, et `GHCR_PAT`, un jeton disposant notamment de la permission `write:packages`. Ces identifiants restent ainsi protégés et ne sont pas stockés dans le dépôt.

Enfin, le workflow `container_workflow` orchestre l'exécution des jobs sur les branches configurées. Le pipeline automatise donc les tests, les contrôles qualité, la construction de l'image Docker et sa publication sur GHCR.

2.3 Gestion des secrets avec Infisical

Les secrets applicatifs (`APP_SECRET`, `AUTH_SECRET` et `ENCRYPTION_KEY`) sont stockés dans Infisical, dans l'environnement correspondant à l'application. Seules les informations de connexion (`INFISICAL_TOKEN`, `INFISICAL_PROJECT_ID` et `INFISICAL_ENVIRONMENT`) figurent dans le fichier `.env`. Celui-ci est exclu de Git grâce au fichier `.gitignore` et du contexte de construction Docker grâce à `.dockerignore`.

Au démarrage, le script `Docker/scripts/entrypoint.sh` vérifie la présence des variables requises et utilise Infisical CLI pour récupérer les secrets au format `.env`. Il les écrit ensuite dans le fichier `.env` de l'application, avec les permissions `0640` et comme propriétaire `www-data`. Apache ne démarre qu'après cette étape. L'application charge ensuite ce fichier avec phpdotenv dans `public/index.php`, ce qui rend les variables accessibles dans la superglobale PHP `$_ENV`. Le token Infisical est supprimé de l'environnement du processus après son utilisation.

Dans CircleCI, les variables configurées correspondent aux paramètres d'accès à Infisical : `INFISICAL_TOKEN`, `INFISICAL_PROJECT_ID`, `INFISICAL_ENVIRONMENT` et, éventuellement, `INFISICAL_DOMAIN`. Elles sont transmises au conteneur avec les options `-e` définies dans `.circleci/config.yml`. En local, le même fonctionnement est assuré par la commande `docker run -env-file .env devsecops-php`.

Ainsi, CircleCI fournit les informations de connexion, Docker les transmet et Infisical fournit

les secrets applicatifs. Ces derniers ne sont pas nécessaires à la construction de l'image et ne sont jamais intégrés à l'image publiée sur GHCR, qui reste générique et peut être utilisée dans plusieurs environnements.

La mise en place a notamment nécessité de corriger le slug de l'environnement Infisical (`INFISICAL_ENVIRONMENT`), de vérifier la région utilisée (Cloud US ou EU) et d'utiliser le service token complet plutôt qu'un simple `INFISICAL_CLIENT_ID`. Un domaine incorrect pouvait notamment provoquer l'erreur `404 Service token not found`.

3 Extension du Pipeline

3.1 Ajout de jobs d'évaluation de code

phpmetrics/phpmetrics

PHP Metrics est un outil d'analyse statique pour PHP qui permet d'évaluer la qualité du code en générant un rapport détaillé. Il mesure des indicateurs tels que la complexité cyclomatique, la couverture des tests, le couplage, ainsi que la taille et le volume des classes et des fichiers. Le rapport généré est sous format html.

Phploc

phploc mesure des aspects tels que la taille du code, le nombre de classes, de méthodes, ainsi que la complexité.

Améliorations possibles :

Ajouter des conditions sur les métriques pour que le pipeline échoue si elles ne sont pas respectées.

3.2 Intégration de la qualité du code

phpmd

PHP Mess Detector scanne le code source et public du projet avec un ensemble de règles prédéfinies. Les résultats sont exportés dans un fichier de rapport au format texte ou html pour permettre leur analyse et leur stockage. Le rapport généré est conservé comme artefact dans la pipeline afin de pouvoir être consulté. Nous avons selectionné les règles suivantes :

- **Cleancode :** Détecte les mauvaises pratiques qui nuisent à la lisibilité et à la maintenabilité du code.
- **Codesize :** Identifie les méthodes et classes trop longues ou trop complexes.
- **Controversial :** Détecte les mauvaise pratiques de programmation.
- **Design :** Analyse la conception du code pour détecter les problèmes d'architecture et de structure.
- **Naming :** Vérifie que les noms des classes, méthodes et variables respectent les conventions de nommage.
- **Unusecode :** Détecte le code inutilisé par exemple les variables, méthodes, fonctions ou encore les paramètres non utilisés.

Sur la figure 1 on peut observer les différentes étapes du pipeline étendu.

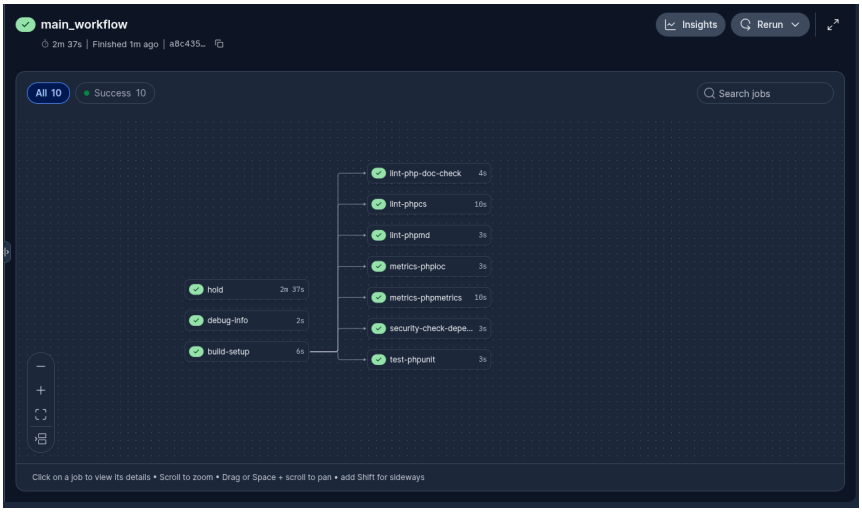


Figure 1: Pipeline étendue (CircleCI)

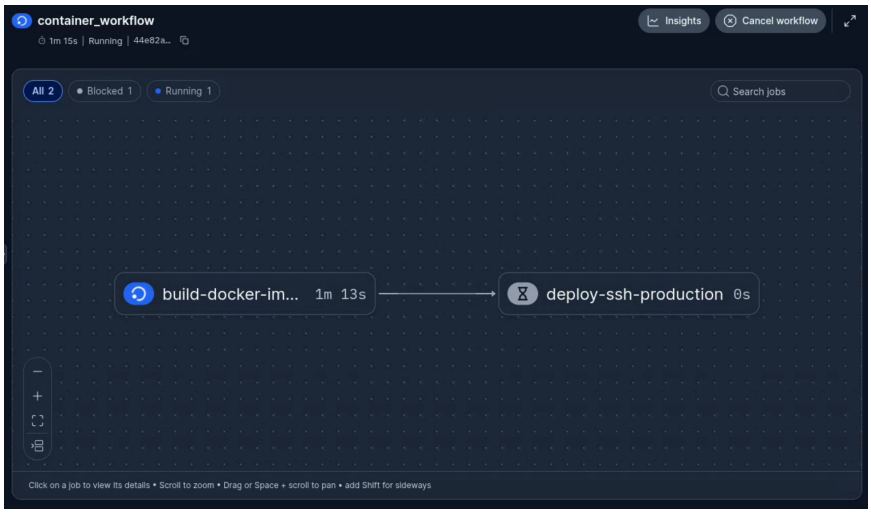


Figure 2: Pipeline de build et de déploiement (CircleCI)

3.3 Déploiement automatisé sur AWS EC2

Nous avons créé une instance AWS EC2 dont l'adresse IP est la suivante : 13.53.206.54. Dans un premier temps, nous avons ouvert à le port 22 (SSH) pour qu'il soit accessible par toutes les adresses IP afin de faciliter l'accès dans le cadre du TP. Par la suite, il sera préférable d'autoriser seulement nos adresses IP pour une meilleure sécurité.

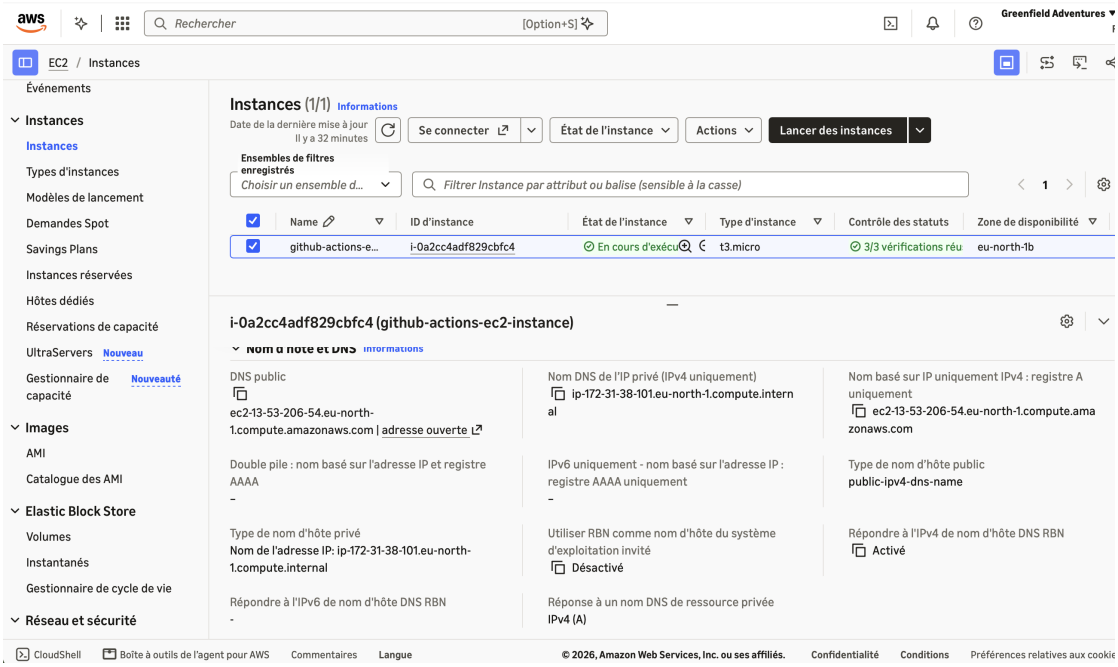


Figure 3: Pipeline container-workflow avec le déploiement sur AWS EC2

Nous nous sommes connectés via SSH avec la commande suivante :

```
chmod 400 "GitHub-action-ec2-key.pem"
ssh -i "GitHub-action-ec2-key.pem" ubuntu@ec2-13-53-206-54.eu-north-1.compute.amazonaws.com
```

On a installé Docker.

Puis nous devons nous connecter au repo github, pour cela on génère une clé SSH depuis l'instance avec la commande suivnte :

```
ssh-keygen -t ed25519 -C "ec2-production-github"
```

Ensuite, nous avons récuperé la clé publique pour la placer sur github (voir figure**??**) afin que l'on puisse avoir accès au répo privé depuis l'instance.

```
sudo mkdir -p /var/www/app
sudo chown -R ubuntu:ubuntu /var/www/app
cd /var/www/app
git clone git@github.com:Bence-dp/DevsecopsPipeline.git .
ssh -T git@github.com
```

Nous sommes à présent authentifié

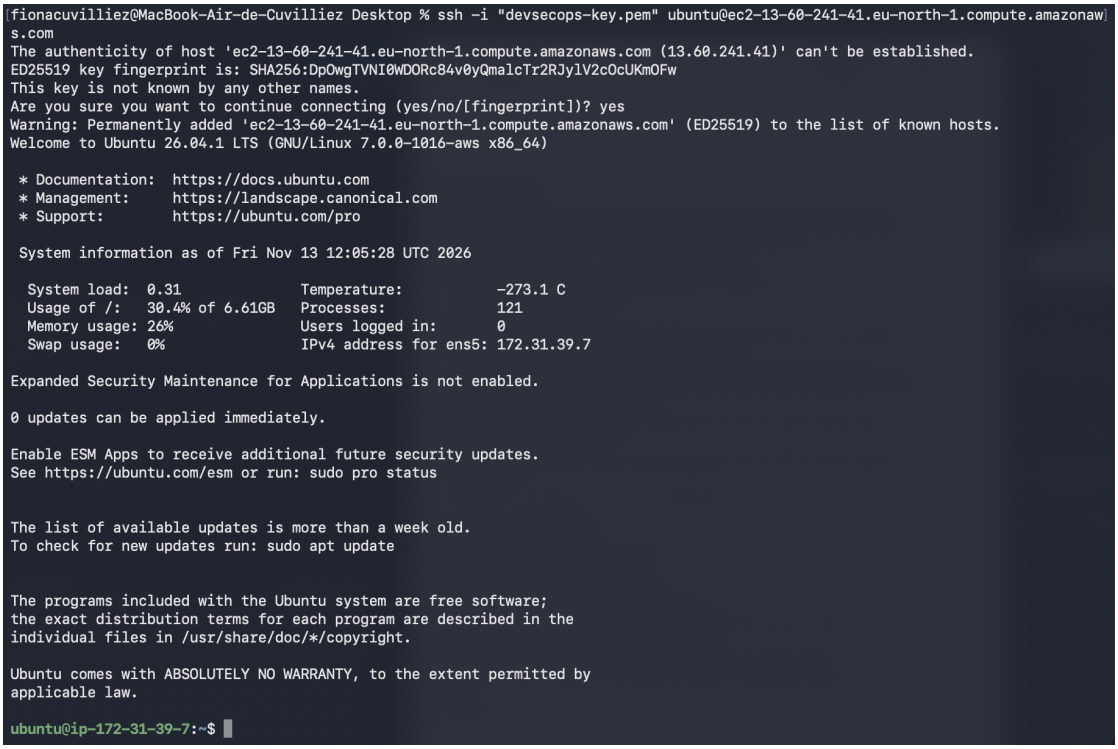


Figure 4: Connexion SSH

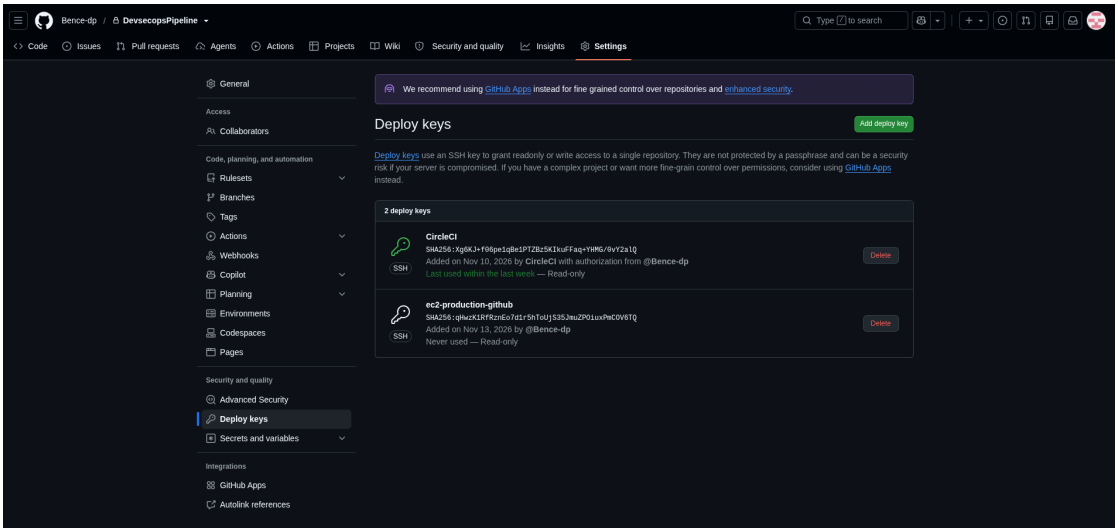


Figure 5: Connexion SSH

```
git remote set-url origin git@github.com:Bence-dp/DevsecopsPipeline.git
git pull origin main

composer install --no-interaction --prefer-dist
```

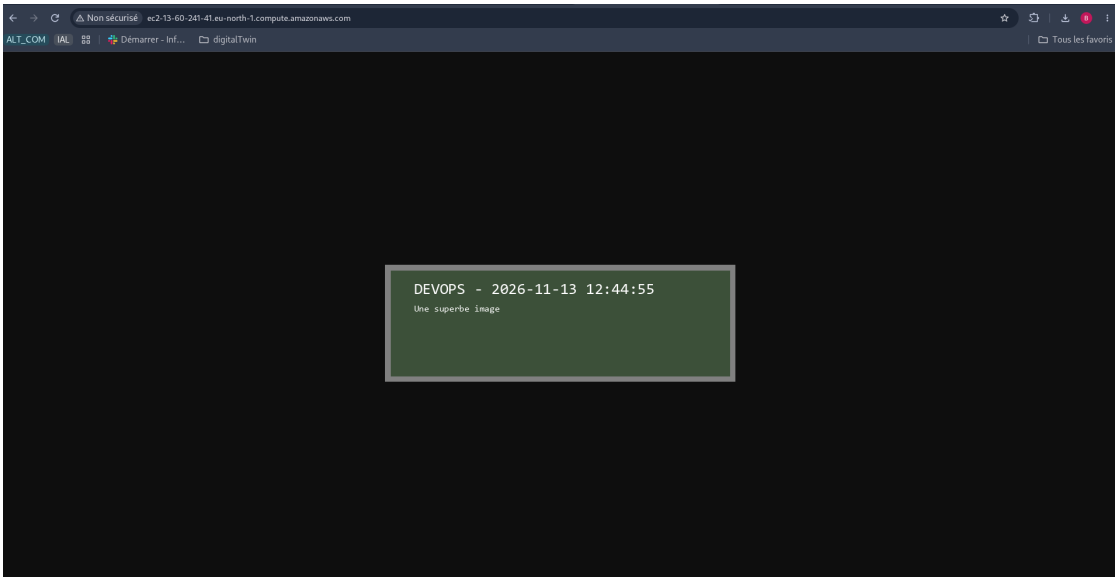
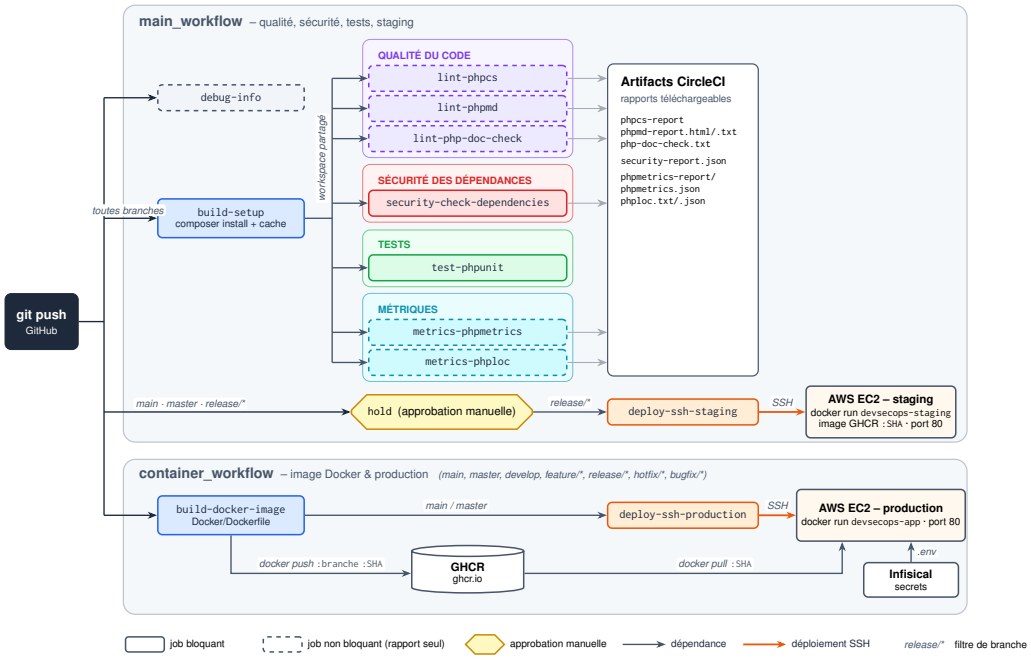


Figure 6: Image

Nous avons sur notre machine, nous avons généré un clé SSH. La clé publique est placée en instance sur le fichier s /.ssh/authorized_keys et la clé privée est ajouté à Circle.

4 Documentation et Rapport

1. **Documentation du processus :** La documentation du pipeline CI/CD se situe dans le document `DOCUMENTATION_CICD.md` situé à la racine du repository.
2. **Rapport final :** Ce document fait office de rapport.
3. **Schéma récapitulatif :** Ce schéma présente les principales étapes et les interactions du pipeline CI/CD ; il répond à la demande d'une représentation claire de son fonctionnement.



4. **Vidéo et script :** La vidéo peut être téléchargée via le lien suivant : `https://www.swisstransfer.com/dl/01a1d6ce-4419-707a-b76e-592aa1c4d87e`

   N'ayant pas pu nous enregistrer vocalement, voici un script de présentation que l'on aurait plus ou moins récité si nous l'avions pu.

   Présentation du pipeline CI/CD et du déploiement Docker

   On commence par effectuer un commit sur la branche `main`. Une fois le commit envoyé sur GitHub, le pipeline CI/CD se déclenche automatiquement.

   Sur le côté, on peut observer deux workflows principaux : le **Container Workflow** et le **Main Workflow**. Ils regroupent les différentes étapes permettant de vérifier la qualité du code, d'effectuer les contrôles nécessaires et de préparer le déploiement de l'application.

   On va commencer par le Main Workflow, qui regroupe plusieurs jobs de vérification.

   Le premier est `lint-php-doc-check`. Il vérifie la présence et la conformité de la documentation PHP, notamment les annotations PHPDoc, afin de faciliter la compréhension et la maintenance du code.

   Ensuite, `lint-phpcs` utilise PHP CodeSniffer pour vérifier que le code respecte les conventions de style PHP définies dans le projet. Il permet notamment de détecter les erreurs de formatage et les mauvaises pratiques d'écriture.

   Le job `lint-phpmd` utilise PHP Mess Detector pour repérer les problèmes de qualité du code, comme les méthodes trop longues, les variables inutilisées ou les structures trop complexes.

   On retrouve ensuite deux jobs consacrés aux métriques du projet. `metrics-phploc` utilise PHPLOC pour mesurer la taille du code source : nombre de lignes, de classes, de méthodes ou encore de fonctions. De son côté, `metrics-phpmetrics` utilise PHPMetrics pour fournir une analyse plus approfondie de la qualité et de la complexité du code.

Le job `phpstan` effectue une analyse statique du code PHP. Il permet notamment de détecter des erreurs potentielles de typage, des appels incorrects ou des problèmes qui pourraient apparaître à l'exécution, sans avoir besoin de lancer l'application.

Le job `security-check-dep` vérifie les dépendances du projet afin d'identifier les éventuelles vulnérabilités de sécurité présentes dans les bibliothèques utilisées.

Enfin, `test-phpunit` exécute les tests automatisés avec PHPUnit pour vérifier que les différentes fonctionnalités de l'application fonctionnent comme prévu et éviter les régressions.

L'ensemble de ces jobs permet donc de contrôler le code avant de passer à la construction et à la publication de l'image Docker.

Une fois les vérifications effectuées, on passe au Container Workflow et à la partie build. Le pipeline construit l'image Docker de l'application à partir du Dockerfile, puis la publie dans les Packages du dépôt GitHub, via GitHub Container Registry, aussi appelé GHCR.

On peut ensuite retrouver l'image directement dans les packages associés au dépôt. Cela permet de centraliser les images Docker et de les rendre accessibles au serveur de déploiement.

Pour comprendre comment tout cela est organisé, on peut ouvrir le fichier `.circleci/config.yml`. C'est dans ce fichier que sont définis les différents jobs, leurs commandes, leurs dépendances et leur ordre d'exécution. On retrouve notamment les étapes de vérification du code, de construction de l'image et de déploiement.

Une fois l'image publiée, le job de déploiement est lancé. Il se connecte au serveur distant, récupère l'image Docker depuis GHCR, puis lance le conteneur correspondant.

Au démarrage, le conteneur récupère également les secrets nécessaires depuis Infisical, comme expliqué précédemment, avant de démarrer Apache et l'application.

On obtient ainsi un processus automatisé : chaque commit envoyé sur `main` déclenche les contrôles qualité et les tests, puis permet de construire, publier et déployer l'application sous forme de conteneur Docker.

L'objectif est de garantir que la version déployée correspond à une version du code ayant passé les vérifications du pipeline, tout en séparant la construction de l'image, la gestion des secrets et son exécution sur le serveur.

5 Sécurité à tous les étages

L'objectif de cette étape est de renforcer la sécurité des différents éléments de notre infrastructure et de notre chaîne d'intégration et de déploiement continus (CI/CD).

5.1 Sécurisation du dépôt Git

Dans un premier temps, il est essentiel de protéger la branche principale (main). Pour cela, plusieurs mesures doivent être mises en place :

- **Interdire les push directs** sur la branche main, afin que toute modification passe obligatoirement par une Pull Request (PR).
- **Définir des rôles et des permissions** notamment le rôle de maintainer, afin de contrôler les personnes autorisées à créer, valider et fusionner les PR.
- **Imposer au moins une approbation** (review approval) avant de pouvoir intégrer une PR dans la branche main.

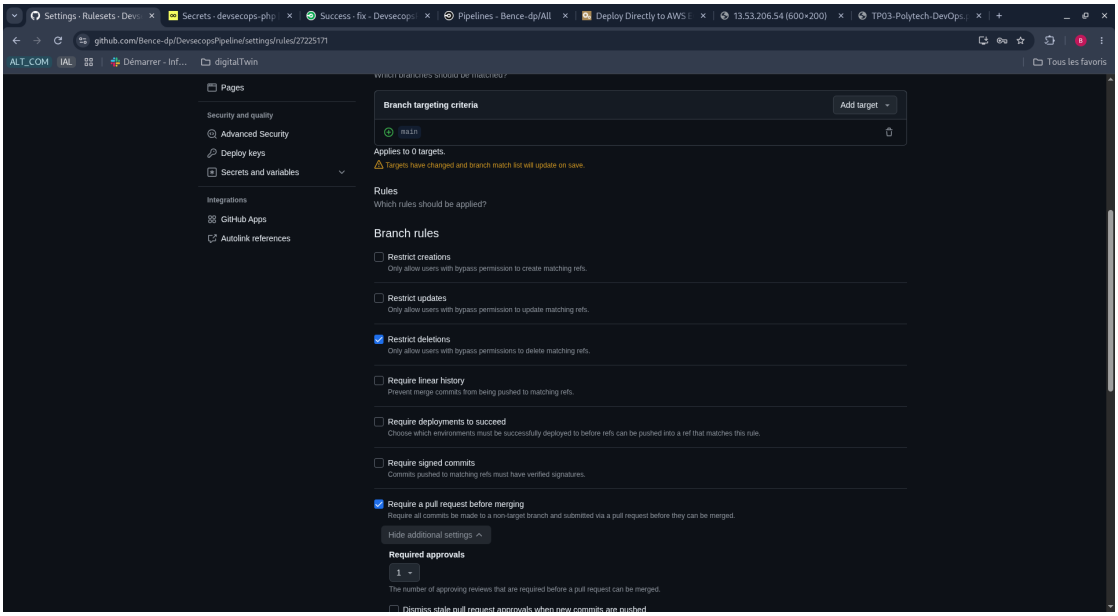


Figure 7: Image

5.2 Vérification de la fiabilité et de la sécurité de l'image Docker

Image docker fiabilité il faut montrer qu'elle soit uniqe. Pour cela, nous avons utiliser : CIRCLE_SHA1 qui permet d'identifier précisément chaque version construite.

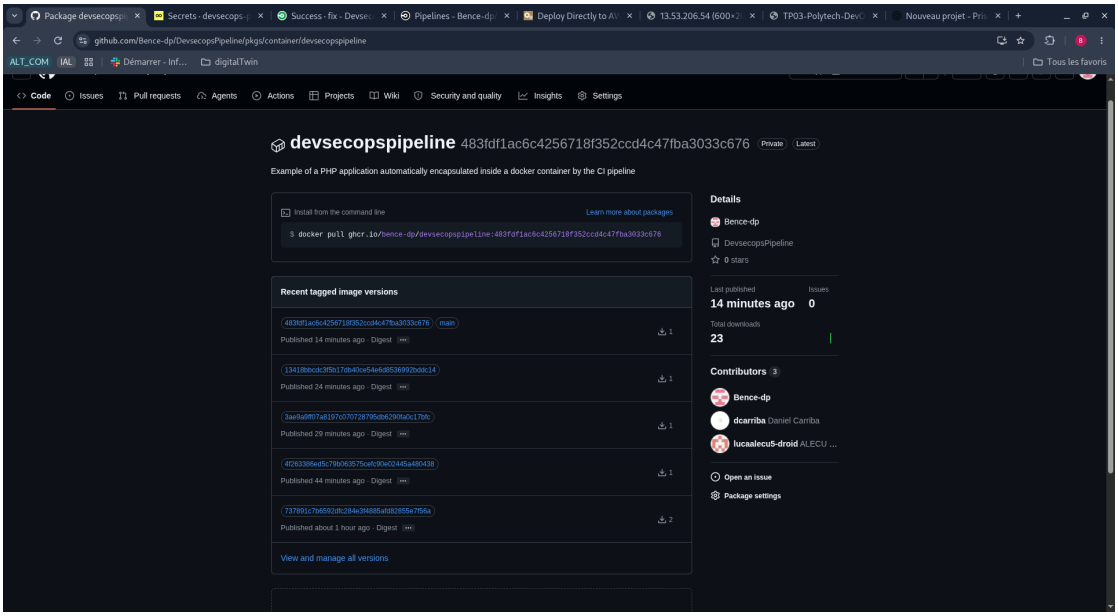


Figure 8: Image

5.3 Sécurisation de l'instance AWS EC2

Notre instance EC2 héberge l'application et constitue le dernier maillon de notre chaîne CI/CD.

Elle doit être protégée contre les accès non autorisés, les attaques réseau et les éventuelles compromissions du système.

Nous sécurisons également l'accès au port SSH. Pour s'y connecter il faut utiliser une clé SSH privée. De plus, nous appliquons le principe du moindre privilège : seuls les accès nécessaires doivent être autorisés c'est pourquoi nous avons limité le port 22 à l'adresse IP des administrateurs.

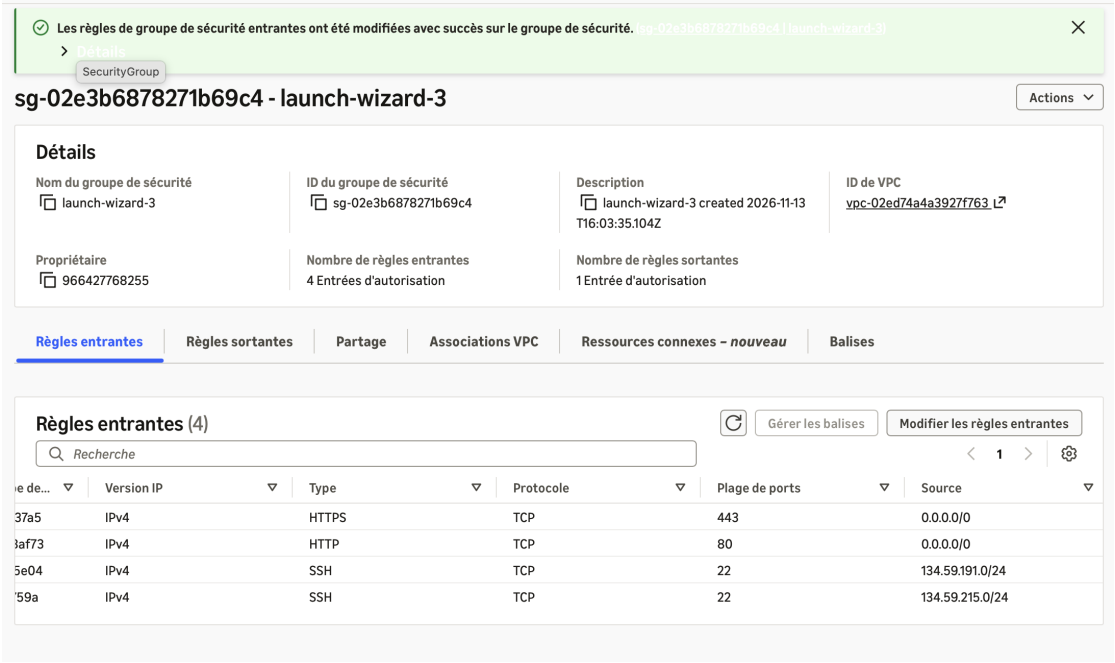


Figure 9: Limitation du port 22

6 Objectifs futurs (bonus)

Cette partie bonus résume les principales pistes d'amélioration de l'infrastructure et du pipeline :

- **Sécurité et suivi :** ajouter un scanner de CVE pour les dépendances et composants, mettre en place un monitoring avec Prometheus et Grafana, et suivre les métriques DORA pour mesurer la performance de la livraison logicielle.
- **Mises à jour :** anticiper la fin de support de PHP 8.2, prévue le 31 décembre 2026, en migrant vers une version maintenue. Pour Apache 2.4.66, assurer une veille et des mises à jour régulières ; la migration vers la version 2.4.71 est envisagée.
- **Optimisation CI/CD :** exécuter en parallèle les jobs indépendants et, pour les étapes séquentielles, commencer par les plus rapides afin de réduire la durée du pipeline et de détecter les erreurs plus tôt.
- **Traçabilité et retours arrière :** relier chaque déploiement à sa branche, sa Pull Request, son issue, son commit et sa version déployée ; documenter les rollbacks et définir une procédure permettant de revenir rapidement à une version stable.